

Examen Parcial de Redes Neuronales y Aprendizaje Profundo

Ph.D. Aldo Camargo

Instrucciones Generales. Este es un proyecto de grupo. Cada pregunta requiere un informe escrito (6–10 páginas) y un repositorio de código reproducible entregado a través de GitHub. Los informes deben incluir: motivación, trabajos relacionados (mínimo 5 referencias), metodología, experimentos, resultados y una discusión crítica de las limitaciones. Se pueden utilizar pesos preentrenados del backbone donde se indique, pero los componentes principales de la arquitectura deben ser implementados por el estudiante. El plagio de cualquier tipo — incluyendo el de cuadernos públicos — resultará en la reprobación automática del curso.

Pregunta 1 — Clasificación de Llantas Dañadas mediante Reconocimiento de Textura

Enunciado del Problema

El desgaste y daño en las llantas de vehículos representa un riesgo crítico para la seguridad vial. La inspección manual es costosa, subjetiva y difícil de escalar en entornos industriales. Los estudiantes deben desarrollar un modelo de clasificación basado en redes neuronales convolucionales (CNN) capaz de distinguir automáticamente entre llantas en **buen estado** y llantas **dañadas o quebradas**, a partir del análisis de imágenes de la textura superficial de la llanta.

El proyecto no se limita a entrenar un clasificador binario: los estudiantes deben explorar al menos **dos arquitecturas diferentes** (por ejemplo, una CNN entrenada desde cero y un modelo preentrenado con fine-tuning como ResNet-50 o EfficientNet), comparar su rendimiento de forma rigurosa, e interpretar las decisiones del modelo mediante técnicas de visualización como **Grad-CAM** — identificando qué regiones de la textura activan la clasificación de daño. Adicionalmente, se debe analizar el impacto del desbalance de clases en el rendimiento del modelo y aplicar al menos una estrategia de mitigación (sobremuestreo, pesos de clase, o aumento de datos orientado).

Conjunto de Datos

- **Tire Texture Image Recognition** — dataset público disponible en Kaggle en la siguiente dirección:
<https://www.kaggle.com/datasets/jehanbathena/tire-texture-image-recognition>.

Contiene imágenes de texturas de llantas organizadas en categorías que permiten distinguir entre llantas en buen estado y llantas con daños visibles. Se recomienda revisar el código de referencia disponible en la misma página como punto de partida, pero la implementación final debe presentar contribuciones propias y no ser una réplica del notebook público.

Entregables Requeridos

1. Implementación y entrenamiento de al menos dos arquitecturas CNN: una diseñada desde cero y una basada en transfer learning con backbone preentrenado en ImageNet.
2. Evaluación comparativa con métricas apropiadas para clasificación potencialmente desbalanceada: exactitud, precisión, recall, F1-score y AUC-ROC. Incluir matriz de confusión comentada.
3. Estudio de ablación sobre al menos dos factores: estrategia de aumento de datos, tasa de aprendizaje, función de pérdida (BCE estándar vs. pérdida focal), o profundidad del backbone.
4. Análisis de interpretabilidad mediante Grad-CAM o una técnica equivalente: mostrar mapas de activación sobre imágenes de ambas clases y discutir qué patrones de textura aprende el modelo a reconocer como indicadores de daño.
5. Análisis del desbalance de clases: cuantificar su efecto en las métricas e implementar y evaluar al menos una estrategia de mitigación.
6. Análisis cualitativo de fallos: identificar y comentar al menos cinco ejemplos mal clasificados, proponiendo hipótesis sobre sus causas.

Pregunta 2 — Segmentación de Vasos Retinianos para Detección de Retinopatía Diabética (U-Net)

Enunciado del Problema

La retinopatía diabética es una de las principales causas de ceguera prevenible en el mundo. El tamizaje temprano depende de la segmentación precisa de la red de vasos sanguíneos en fotografías de fondo de ojo (imágenes de fundus) — un problema de predicción densa en el que el modelo debe asignar una etiqueta de primer plano o fondo a cada píxel. Los estudiantes deben implementar una U-Net (o una variante mejorada como Attention U-Net o U-Net++) para realizar segmentación binaria píxel a píxel de los vasos retinianos.

Más allá de obtener una buena segmentación en un único benchmark, los estudiantes deben analizar cómo el rendimiento del modelo se degrada al evaluarlo en imágenes de una **fente clínica diferente** (cambio de dominio), y deben proponer e implementar al menos una estrategia de mitigación. Las alternativas posibles incluyen preprocesamiento con ecualización de histograma o CLAHE, aumento de datos en tiempo de prueba (test-time augmentation), o ajuste fino sobre un pequeño subconjunto del dominio objetivo.

Conjuntos de Datos

- **DRIVE** (Digital Retinal Images for Vessel Extraction) — 40 imágenes de fundus con máscaras manuales de vasos y una división fija de entrenamiento/prueba. Benchmark de referencia estándar. Disponible en <https://drive.grand-challenge.org/>.
- **STARE** (Structured Analysis of the Retina) — 20 imágenes con dos conjuntos de anotaciones de expertos, lo que permite realizar un análisis de concordancia entre anotadores. Disponible en <https://cecas.clemson.edu/~ahoover/stare/>.
- **CHASE_DB1** — 28 imágenes adquiridas con una cámara de fundus diferente, ideal para el experimento de generalización entre conjuntos de datos. Disponible en <https://blogs.kingston.ac.uk/retinal/chasedb1/>.

Entregables Requeridos

1. Implementación personalizada de U-Net en PyTorch (sin copiar código de bibliotecas de segmentación existentes). El backbone debe implementarse desde cero o adaptarse con modificaciones documentadas.
2. Estudio de ablación que compare al menos dos decisiones arquitectónicas o de entrenamiento (por ejemplo, profundidad de la red, diseño de las conexiones de salto, función de pérdida: BCE vs. Dice vs. combinada).
3. Evaluación en distribución sobre DRIVE (sensibilidad, especificidad, F1, AUC-ROC).
4. Experimento de generalización entre conjuntos de datos: entrenar en DRIVE, evaluar en CHASE_DB1. Reportar la brecha de rendimiento y analizar su causa.
5. Análisis cualitativo de fallos: identificar qué tipos de vasos (capilares finos vs. arterias grandes) son más difíciles de segmentar y explicar por qué desde una perspectiva arquitectónica.
6. Implementación de una estrategia de adaptación de dominio o preprocesamiento, con evidencia de su efecto sobre la brecha entre conjuntos de datos.

Pregunta 3 — Detección de Cumplimiento de Normas de Seguridad en Obras de Construcción (YOLO)

Enunciado del Problema

En obras de construcción activas, los trabajadores deben usar Equipos de Protección Personal (EPP) que incluyen cascos, chalecos de seguridad y, en algunos contextos, arneses. Los estudiantes deben entrenar y evaluar un modelo YOLO (se recomienda YOLOv8 o YOLOv9) para detectar trabajadores y clasificar su estado de cumplimiento del EPP en imágenes y secuencias de video cortas. La tarea va más allá de la detección de objetos estándar: los estudiantes deben manejar **oclusión severa** (trabajadores parcialmente

ocultos detrás de maquinaria o andamios), **detección de objetos pequeños** (trabajadores lejanos que aparecen con baja resolución) y **desbalance de clases** (los trabajadores no conformes son típicamente escasos en las grabaciones reales).

Sobre el modelo de detección, los estudiantes deben implementar un **verificador de cumplimiento basado en reglas**: un módulo de posprocesamiento que recibe las detecciones por fotograma y genera una tasa de cumplimiento por fotograma, marcando los bounding boxes clasificados como no conformes. El informe final debe incluir una discusión sobre las implicaciones éticas de implementar dicho sistema, abordando específicamente las consecuencias de los falsos negativos (violaciones no detectadas) frente a los falsos positivos (marcar incorrectamente a un trabajador que sí cumple las normas).

Conjuntos de Datos

- **Safety Helmet Detection Dataset** — aproximadamente 5,000 imágenes con anotaciones de bounding box en formato YOLO, disponible en Roboflow Universe (<https://roboflow.com>). Usar como conjunto principal de entrenamiento. Licencia: CC BY 4.0.
- **PPE Detection Dataset** — aproximadamente 10,000 imágenes que cubren cascos, chalecos y guantes, también disponible en Roboflow Universe. Usar para aumentar los datos de entrenamiento o para evaluación entre conjuntos de datos.
- **SHWD** (Safety Helmet Wearing Dataset) — disponible en GitHub ([njvisionpower/Safety-Helmet-Wearing-Dataset](https://github.com/njvisionpower/Safety-Helmet-Wearing-Dataset)). Contiene ejemplos negativos explícitos de “sin casco”, fundamentales para entrenar un clasificador de cumplimiento confiable.

Entregables Requeridos

1. Modelo YOLO ajustado con decisiones de entrenamiento completamente documentadas (configuración de anclas, estrategia de aumento de datos, activación o desactivación del aumento mosaico, resolución de entrada).
2. Evaluación cuantitativa: mAP@0.5 y mAP@0.5:0.95 sobre el conjunto de prueba, con desglose por clase (trabajador con casco, trabajador sin casco, chaleco, sin chaleco, etc.).
3. Análisis de robustez ante oclusión: particionar el conjunto de prueba de forma manual o semiautomática según el nivel estimado de oclusión (bajo / medio / alto) y reportar la caída de rendimiento entre grupos.
4. Implementación del verificador de cumplimiento basado en reglas con demostración: una secuencia de fotogramas anotados o un clip de video corto que muestre el sistema funcionando sobre imágenes de obras no vistas durante el entrenamiento.
5. Sección de implicaciones éticas (mínimo una página): abordar el balance entre falsos negativos y falsos positivos, las preocupaciones sobre la privacidad de los trabajadores y los posibles sesgos introducidos por la distribución de los datos de entrenamiento.

Fecha Límite y Formato de Entrega. Informe (PDF, formato NeurIPS) + enlace al repositorio de GitHub entregados a través de la plataforma del curso. Las entregas tardías serán penalizadas con un 10 % por día. El código debe ejecutarse de extremo a extremo con un único comando documentado en el README, sobre una especificación de entorno proporcionada (`requirements.txt` o `environment.yml`). **Fecha de entrega y presentación: 30 de Mayo del 2026, en clase.**